CISC 322 / ASSIGNMENT 1

WezTerm

Conceptual Architecture

Prepared by Arch Nemesis

PROJECT TEAM

Paige Brossard
23mfq2@queensu.ca

Kezia Sorensen
23hjl2@queensu.ca

Sandy Khodak
23nr52@queensu.ca

Sieena Bragg
23dlc4@queensu.ca

Ali Haider Cheema
20ahc1@queensu.ca

Stephanie Douglas
23pfc1@queensu.ca

Contents

1 Introduction **3**

2 WezTerm Architecture Overview **5**
2.1 Overall Architectural Principles 5
2.2 Subsystems 5
2.2.1 GUI/Input 5
2.2.2 Multiplexer 5
2.2.3 Terminal Emulation 6
2.2.4 Session Connections 6
2.2.5 Configuration 6
2.3 Dependencies 6
2.4 Control Flow 7
2.4.1 Session Establishment 8
2.4.2 Keyboard input and program output 8
2.4.3 Creating a split pane 9
2.4.4 Concurrency and communication boundaries 9
2.4.5 Performance characteristics 9
2.5 Division of Responsibilities 9
2.6 Evolution of Wezterm 10

3 External Interfaces **10**

4 Use Cases **11**
4.1 Use Case 1: Connecting to a Remote Multiplexer 11
4.2 Use Case 2: Editing the Configuration 12

5 Conclusion **13**

6 Lesson Learned **13**

7 Data Dictionary **14**

8 Naming Conventions **14**

9 References **15**

10 AI Collaboration Report **16**

Abstract

WezTerm provides local and remote terminal access from a single application; however, its flexibility is determined by how well the different areas of responsibility are coordinated. As such, this report investigates these relationships to assist developers and technical managers in evaluating the ramifications of architectural modifications. We have identified five conceptual subsystems based upon available public documentation, release notes, and project discussions: GUI/Input, Multiplexer, Terminal Emulation, Session Connections, and Configuration.

We have grouped related responsibilities together, created dependency maps, and illustrated interaction patterns using sequence diagrams for both remote multiplexer reconnection and automatic configuration reloading. Our model distinguishes session organization, terminal interpretation, and connectivity, with Configuration being an influence on each of the other four subsystems.

Two architectural styles are used to illustrate the benefits and challenges. Event-driven control supports user actions and independent program output but complicates state coordination and testing. In standalone multiplexer configuration, a client/server relationship enables sessions to continue even when the GUI is disconnected, as long as the server and program(s) continue to operate, however, it presents challenges for communication, deployment and reconnecting. Our analysis has identified shared interfaces as a point where coordination can occur among developers, and has identified output processing, scrollback storage, and remote communication as areas that may have implications for performance. The results of our analysis can be utilized as a basis for assessing future changes and dividing development tasks.

However, our boundaries are conceptual only at this time, not actual source-module boundaries that have been verified. Performance concerns were inferred, not benchmarked. Further analysis of the implementation will be required to validate our assumptions.

1 Introduction

Purpose

In this report, our group examines the conceptual software architecture of WezTerm, a cross-platform terminal emulator and multiplexer written in Rust and configured using Lua. Standard terminal emulators simply run command-line shells inside a single desktop window. A terminal multiplexer goes further. It lets users create, split, and manage dozens of active terminal sessions at once, including remote sessions across various servers. WezTerm merges both roles into one tool. That combination makes it an interesting case study for how open-source systems divide complex operational responsibilities over time.

We wrote this report primarily for software developers who want to understand WezTerm's high-level layout, subsystem roles, and internal control flow. We want to make clear that this is a *conceptual* architecture model. We derived our model from WezTerm's public documentation, release notes, and GitHub discussions, not a direct assessment of its Rust source code. We do not imply that the codebase contains files with these exact names.

We divided WezTerm into five components that manage the following responsibilities: GUI/Input, the Multiplexer, Terminal Emulation, Session Connections, and Configuration. GUI/Input draws windows, tabs, and panes as well as catches mouse and keyboard events. The Multiplexer organizes said panes into tabs, windows, workspaces, and network domains. Terminal Emulation turns ANSI output into a text grid and converts keystrokes into input for applications. Session Connections manages the data pipes to local programs (PTY or ConPTY) and remote systems, which we will discuss later. Configuration runs the user's Lua script to supply shortcuts, themes, and connection rules to the other subsystems.

In our dependency model, directed arrows indicate where one subsystem relies on data or behavior from another. The GUI depends on the multiplexer to know how sessions are arranged, and it relies on terminal emulation for rendered screen state. The multiplexer relies on terminal emulation to track pane buffers and on session connections to establish live streams. Terminal emulation utilizes active session connections to exchange raw bytes with running processes. Configuration acts as a cross-cutting provider across all four other subsystems.

Derivation Process

Initially, we researched WezTerm through the official documentation available on wezterm.org, "What is a terminal" page, the demo video, and release notes and discussions on GitHub. Since all these resources provide information about the functionality rather than architecture, we compared them and classified the responsibilities that appeared similar into five conceptual subsystems: GUI/Input, Multiplexer, Terminal Emulation, Session Connections, and Configuration. We then created use cases based on our research, such as connecting to a remote multiplexer and changing configuration, and created sequence diagrams where we depicted the flow of actions made by the user through the subsystems. Afterward, we defined the subsystems involved in each use case and added them to our dependency diagram, where we drew a directed arrow wherever one subsystem relies on data or behaviour from another. In sum, all these dependencies and interactions gave us the information about event-driven control flow and client/server architecture in case when a standalone multiplexer is used. Since we started our research using documentation rather than code, our boundaries of subsystems are conceptual and will be verified against the implementation in A2.

Control Flow and Concurrency

System activity in WezTerm is guided by user keystrokes, layout commands, and asynchronous program output. When a user presses a key, WezTerm checks it against configured key assignments first. If there is a match, it executes the action right away, such as splitting a pane. If no binding matches, it encodes the keypress and pushes those bytes down the active pane's session connection to the child process. Output from that process travels back along the exact same channel, prompting terminal emulation to update screen state and triggering a GUI repaint.

Multiple panes can execute simultaneously, each managing its own background connection and process tree. When running with a dedicated multiplexer server, the GUI client and multiplexer server operate as separate OS processes. Critical performance areas include high-throughput terminal output, memory overhead from deep scrollback buffers, and network latency in remote domains.

Evolution and Development

Wez Furlong started the project back in December 2017. Since then, WezTerm has grown into a major open-source project backed by hundreds of community contributors. It began simply as an escape-sequence parser with basic rendering abilities before adding Lua scripting, search and copy modes, and multi-domain multiplexing over SSH and TLS Development heavily reflects community feedback, with contributions welcomed across bug reports, feature requests, design discussions, and documentation fixes.

Organization of the Report

The rest of this report is organized sequentially:

- Architecture: Straightforward breakdown of subsystem roles, dependency relationships, and global control flow.
- Development & Evolution: Developer division of labour and historical system growth.
- Diagrams & Use Cases: External interface definitions, sequence diagrams for core use cases, a data dictionary, and naming conventions.

- Conclusions & Reflection: Key findings, lessons learned, and formatted references
- AI collaboration report:

Salient Conclusions

- Decoupled Architecture: Separating graphical presentation from session management allows local and remote sessions to look and feel completely identical to the end user.
- Isolated Responsibilities: Mapping GUI, multiplexing, emulation, connection streams, and configuration to distinguishable roles makes the system significantly easier to maintain and test.
- Community-Guided Growth: An open, discussion-driven development process directly dictates how new features and protocol expansions get added over time.

2 WezTerm Architecture Overview

2.1 Overall Architectural Principles

WezTerm is designed as a collection of cooperating subsystems that separate the user interaction, terminal management, session connectivity, and configuration responsibilities. The primary subsystems are GUI/Input, Multiplexer, Terminal Emulation, Session Connections, and Configuration. [2] This organization allows WezTerm to support both basic local terminal use and more complex configurations involving multiple panes, workspaces, and remote sessions. [2][3]

A key architectural principle is the separation between terminal organization and graphical representation. The multiplexer manages the structure and organization of terminal environments. This includes windows, tabs, panes, workspaces, and domains. [3][11] The GUI/Input subsystem provides the user facing interface in which these environments are controlled. Because the multiplexer can operate independently of the graphical interface, terminal environments can remain active without a directly connected GUI.[2][8]

The architecture also separates the management of terminal state from the processes that produce and receive terminal data. Terminal emulation interprets terminal output and maintains display states, while session connections provide access to the local or remote processes through mechanisms such as pseudo-terminals and SSH. [4][6] This separation allows different session types to use consistent terminal interfaces. [9]

Together, these divisions improve maintainability and flexibility by separating responsibilities. Configuration influences multiple subsystems, allowing behavior like key bindings, domains, workspaces, and rendering to be customized using Lua. [1][12][16]

2.2 Subsystems

2.2.1 GUI/Input

The GUI/Input subsystem controls the interaction between the user and WezTerm. It manages graphical windows, keyboard and mouse events, and presents terminal information. [12][16] User actions such as changing tabs, creating panes, or entering commands are translated into operations on the underlying terminal environment. Key and mouse bindings can be configured to control these actions. [12][13][14]

2.2.2 Multiplexer

The Multiplexer organizes terminal sessions through windows, tabs, panes, workspaces, and domains. [3][10][11] It receives requests from the GUI/Input subsystems, and the coordinates creation, organization, and terminal environment selection. It can also interact with the session connections when terminal sessions or processes need to be created in a domain. [17]

2.2.3 Terminal Emulation

The Terminal Emulation subsystem interprets the data exchanged between WezTerm and terminal processes. It processes terminal control and escape sequences to maintain the current terminal state, including screen contents and scrollback. [4][6][15] Output from a process is interpreted into display state, while user input is passed toward the appropriate session. [4]

2.2.4 Session Connections

The Session Connections subsystem manages the connection between WezTerm and the environment in which the terminal process is running. Local sessions communicate with processes through pseudo terminals, while remote sessions can use SSH. [4][9] WezTerm can also connect to remote multiplexer environments, allowing sessions to remain available independently of the local GUI. [3][9]

2.2.5 Configuration

The Configuration subsystem defines the settings that control how WezTerm and its other subsystems behave. Users can configure key and mouse bindings, default domains, workspaces, launching behavior, and rendering settings using Lua. [12][17]

2.3 Dependencies

We constructed the conceptual model based upon documentation regarding WezTerm's terminal behaviors, multiplexing capabilities, configuration options, and session connections [1-4]. We grouped similar responsibilities together in five subsystems and then determined how each responsibility was dependent on others. For instance, GUI/Input depends on session organization provided by Multiplexer, and the interpreted screen state from Terminal Emulation. The documentation of the Pane abstraction covers multiple areas of concern; therefore, our boundaries are analytical and not necessarily tied directly to source module divisions [10]. Internal scheduling and exact implementation boundaries will be determined in A2.

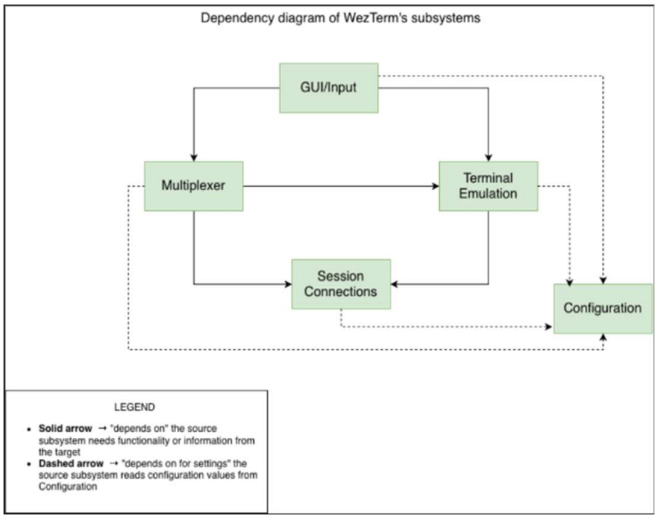


Figure 1: How WezTerm's core components depend on one another for runtime state, operational data, and configuration settings.

The GUI/Input component depends upon Multiplexer for displaying the structure of terminal sessions. WezTerm defines a multiplexing domain as a unique collection of windows and tabs. WezTerm automatically creates a local domain at launch time and allows users to start or connect to additional domains. When connected to a domain, the GUI can attach its windows and tabs to the native interface [3]. Multiplexer also defines how panes are arranged within tabs and maps

MuxWindows to workspaces. Each MuxWindow has an associated workspace label. The GUI displays only those windows that belong to the active workspace [11]. This dependency enables the GUI to visualize and manipulate the structure of terminal sessions.

The GUI/Input component depends on Terminal Emulation for displaying the interpreted terminal state. Terminal programs emit printable text and escape sequences to position the cursor, modify the contents of the screen, and define display attributes [4]. Terminal Emulation maintains a parsed representation of the state associated with each pane (e.g., the parsed screen and scrollback) [10]. The GUI/Input component displays that state to the user. The Multiplexer component depends on Terminal Emulation for maintaining the terminal state for its panes. WezTerm defines the Pane abstraction as tracking a pseudo-terminal (or real serial terminal), the associated processes, as well as parsed screen and scrollback [10]. Therefore, the Pane abstraction defined by WezTerm occupies a position in the Multiplexer's session structure and has access to a live terminal's state. Our conceptual separation of these concerns helps to explain their relationship; however, the Pane abstraction described by WezTerm spans both concerns.

The Multiplexer component depends on Session Connections to create or join sessions under the selected domain. For local Unix sessions, WezTerm allocates a pseudo-terminal (PTY) and launches the selected child program attached to the PTY client side [4]. Connections to another multiplexer use Unix domains via Unix sockets, SSH domains via an SSH channel, or TLS domains via a TCP connection secured with TLS. SSH multiplexing domains require a compatible installation of WezTerm on the remote host [3]. All of these connection types provide session environments (either local or remote) for WezTerm's session structure represented by the GUI.

The Terminal Emulation component uses Session Connections to communicate with the program associated with a pane. It encodes completed user input and transmits it through the terminal connection to the program. It interprets data received over that connection into terminal state [4], [13]. Terminal Emulation relies on Session Connections even though data flows bidirectionally because Session Connections provide an established communication medium. The PTY interface is provided by the operating system, and the shell (or any other program running) exists outside of WezTerm. Sessions on Windows using ConPTY utilize a different platform implementation [4].

The Configuration component provides values that many parts of the application use. The GUI/Input component references configuration for key and mouse bindings as well as visual options. The Multiplexer and Session Connections components read settings governing domain selection and program selection. The Terminal Emulation component references options governing terminal emulation functionalities such as keyboard encoding [1], [3], [13], [17]. WezTerm executes a Lua configuration script and expects it to return a configuration table. The program monitors the file and reloads it automatically whenever it changes. Most options become effective immediately upon modification. Options specified via command line overrides take precedence over those specified in the file [1]. Configuration influences many aspects of WezTerm without constituting a distinct step in processing terminal input or output.

2.4 Control Flow

The global control flow in WezTerm receives user input, responds to requests to modify session layout, and receives output from programs executing within panes. Control is transmitted differently depending upon the type of interaction. The GUI/Input component receives an operating system event and either initiates an assigned WezTerm action or transmits completed input for encoding and transmission to the active pane's program [12], [13]. Session actions operate on WezTerm's session structure through Multiplexer. Output from programs is received through the terminal connection and interpreted into terminal state to be rendered by GUI/Input. In cases where a separate multiplexer server is configured, communication with that server involves a process boundary crossing via the chosen domain connection [3].

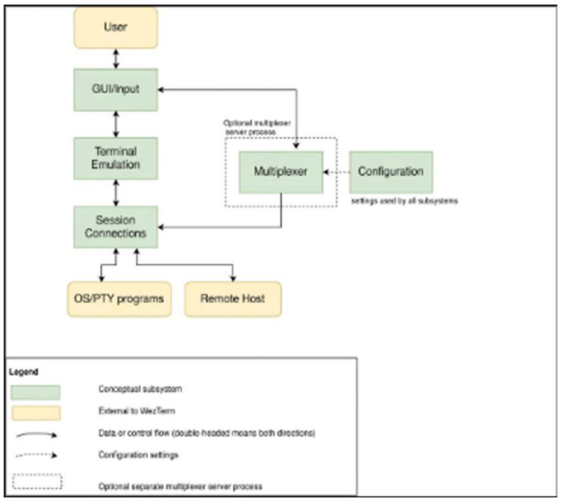


Figure 2: Data paths connecting WezTerm's internal components to local shell processes and remote host connect

2.4.1 Session Establishment

WezTerm allocates a PTY and launches the selected child program attached to the PTY client-side. WezTerm communicates with the program through the PTY controller side. The controller side of a PTY also allows WezTerm to communicate terminal size updates. WezTerm typically utilizes a shell as the child program, although it can execute other programs [4].

For local sessions, default_prog can specify the program and arguments to utilize. If default_prog is not configured, WezTerm utilizes the default shell selected for the present platform. On POSIX-based systems, WezTerm utilizes the shell specified in the password database for the present user. On Windows-based systems, WezTerm utilizes the COMSPEC environment variable if it exists; otherwise, cmd.exe is utilized [17]. Windows-based sessions initiated using ConPTY utilize the Windows pseudo-console mechanism and not a Unix PTY [4]. It should not be assumed that default_prog and local logic for selecting a default program determine the shell on remote hosts.

2.4.2 Keyboard input and program output

Operating systems report key presses as key events, and GUI/Input receives them for processing. WezTerm's key handling described in documentation covers composing input using an Input Method Editor (IME) if enabled and handling dead keys where appropriate. Key assignments in WezTerm can match physical keys, raw key codes, or keys as mapped by the operating system's keyboard layout. Once WezTerm identifies a matching key assignment, it executes the configured action [12].

The terminal will not necessarily receive input until its composition is finished [12]. Once input is finished and ready for delivery to the terminal, and if it is not handled by a matching assignment, WezTerm will encode it as bytes using the active terminal keyboard encoding and write them to the terminal connection for the active pane. The documented default encoding is compatible with xterm. Both configuration and supported protocol requests can modify this encoding [13].

For a pane attached to a local Unix PTY, the input travels through the PTY to the program. If that program is a shell, it will interpret a command and possibly launch another program. That program may read input and write output directly through the terminal interface; the shell does not usually relay that program's terminal input or output [4].

Output returns through the terminal connection back to Terminal Emulation. Text and escape sequences are parsed, and the pane's screen state is updated [4], [10]. Lines that scroll off the visible screen can be saved in scrollback [15]. GUI/Input renders the terminal content. Output can proceed independently of subsequent keyboard input (such as output from an executing program providing

progress updates).

2.4.3 Creating a split pane

Key assignments are capable of invoking SplitPane as a WezTerm action. GUI/Input recognizes the assignment and initiates the action instead of sending the key combination as standard input to the terminal. SplitPane creates a new pane in the specified direction: Up, Down, Left, or Right. Its command field specifies the program to run in that pane; if no command is provided, the default program is utilized [12], [14].

Multiplexer coordinates altering the tab's set of panes in our conceptual model; Session Connections opens up a new session, and Terminal Emulation tracks the new pane's terminal state. GUI/Input displays the modified layout and content. The existing pane retains its own session connection and terminal state, while the new pane receives a separate session.

2.4.4 Concurrency and communication boundaries

Multiple panes can maintain active terminal sessions simultaneously. Each pane tracks its terminal connection, associated processes, parsed screen, and scrollback buffer [10]. Normal keyboard input is directed towards the active pane, but programs executing in other panes may continue to execute and generate text output. A pane may be associated with multiple processes (such as a shell and a program it executed); a pane should not be assumed to correlate with precisely one operating system process [4], [10].

When connecting to a stand-alone multiplexer server, the GUI operates as a separate process; both GUI and multiplexer server may reside on different machines. Unix domains communicate via Unix socket connections. SSH domains utilize SSH as their connection channel to a remote WezTerm multiplexer. TLS domains utilize a TCP connection secured with TLS. Connections utilizing TLS can be established via SSH [3]. Utilizing a separate multiplexer allows session management to persist even when disconnected from a GUI, provided that both multiplexer server and sessions remain active. A typical SSH connection alone does not afford this independence [3].

The interactions described above support an event-driven model of control flow. Events from users trigger input processing or session activity. Program output received triggers modifications to terminal state. Configurations with separate multiplexer servers exhibit client/server relationships between GUI and session servers as well. The units of concurrency identified in this model are active program processes and, when configured, separate GUI and multiplexer processes. The number of internal threads utilized by these units and their scheduling policies are not specified in documentation reviewed.

2.4.5 Performance characteristics

The output path has a significant impact on performance because it requires parsing terminal output, integrating it into the current screen state and then drawing that output while maintaining responsiveness in the GUI. The selection of the front_end configuration option in WezTerm (OpenGL, WebGpu or Software) will determine what type of rendering method is employed. Both OpenGL and WebGpu employ hardware acceleration to create images from the raster data; whereas, Software employs only CPU-based methods of creating those images [16]. Increasing the scrollback_lines value will result in increased memory usage. This may become an issue if there are multiple long-lasting tabs open [15]. Remote multiplexing adds communication delays between user inputs and their corresponding responses. The responsibilities listed above show potential sources of time lags and/or resource pressures, as opposed to empirically determined performance bottlenecks.

2.5 Division of Responsibilities

Wezterm is now a massive project, but it started with just Wez Furlong [5]. Wezterm welcomes many types of contributions including creating documentation, suggesting or creating new

features/updates, or bringing bugs to Wezterm's attention [19]. They welcome contribution from people who aren't involved on the coding side of things too like design ideas, typos that've been found, and ideas to improve their docs [21], [22]. There are currently 1.5k issues in GitHub, 5.6k users, and 236 contributors [20].

Wezterm welcomes all forms of contribution and gives plenty of tips on getting started, testing, and submitting a pull request [19]. Since anyone can contribute, there are plenty of commits by different contributors, and even in the change log, they frequently link features, upgrades, and fixes to users [23]. There are many ways to find areas in need of help, for example, How can the community help support WezTerm's next steps? is only one discussion where bew discusses PR triage and a tracker, specific ways to help like testing PRs and reporting it were also mentioned [18]. There are also plenty of discussions for different questions and bugs, and people actively answer each other's questions and support each other when they can [21].

These conceptual responsibilities suggest how development work could be divided among developers or teams. As shown in the report, the GUI/Input handles the interaction between the user and Wezterm, the multiplexer organizes sessions, terminal emulation interprets data between Wezterm and the terminal processes, session connections handle connections, and configuration handles the settings. While these subsystems are separate in certain ways, they rely on each other for different things, like the terminal emulation uses session connections to communicate with programs, and there would have to be some collaboration or communication to make sure that both parts can work together so that the terminal emulation can communicate with programs. There would need to be some form of coordination for both this example and many other parts of Wezterm to make sure that these subsystems can work together.

2.6 Evolution of Wezterm

The project began on Dec 7th, 2017, and was initially intended to be used as a terminal escape sequence parser and differential screen protocol with textual and GUI rendering [5]. It was supposed to be able to run as both a textual terminal multiplexer and GUI terminal emulator [5]. Initial work on the project included: work on scrollback, and running it on Linux with XCB and later added OpenGL ES 2 [5]. Something interesting to note is that projects that were being worked on would be updated very frequently, for example OpenGL ES 2 became OpenGL ES 3 and rendering underline, italic, bold, strikethrough added double-underline [5]. They asked for feedback very early on in the process like raising the priority for things that haven't been implemented yet [5].

In later commits, some significant updates were adding lua based configuration, they added modes, like copy and search mode, they added shortcuts like ctrl-shift-u to activate a character picker modal overlay, etc [23]. When fixing, improving or adding things, they added a comment about the person who mentioned the new feature, improvement, or problem [23]. A lot of what they do is in response to feedback [23]. Currently, Wezterm has many features including having multiple windows, being able to split it and move between panes, and different ways to connect to local/remote multiplexers [2].

3 External Interfaces

Graphical User Interface

Wezterm runs on Linux, macOS, NetBSD, Windows and FreeBSD [2], and uses each platform's own windowing system, such as X11 or Wayland on Linux [1] and Win32 on Windows [3]. When users type, scroll or paste text, WezTerm either handles the input itself, like opening a new tab, or converts it into escape sequences and sends it to the running program [4]. The GUI displays what the program sends back. Some of the many examples would be text that appeared to be style, images, window and links, which then open in the system's web browser. One thing to remember, the windows

and tabs the GUI shows are the ones managed by a "domain." This is the local domain however the GUI also attaches to domains run by a local or remote multiplexer [3].

Files

When WezTerm starts, it loads its settings from a configuration file called wezterm.lua. A Lua script containing settings like altering different appearances. The terminal has this file automatically run and reload when there are alterations. If there happens to be an error within the file, WezTerm displays a message and uses its default settings. The settings that are entered on the command line take over (override) the file [1].

Database

WezTerm's documentation explicitly mentions a database of terminal capabilities called termcap. This database was created to map different terminal functions, such as switching to bold rendering, to the exact codes needed to trigger them [4]. Although WezTerm does not have a database of its own, it keeps its settings in files like wezterm.lua [1]. However, programs running inside WezTerm rely on the operating system's terminfo database, which is the successor to the older termcap. Programs read the environment variable (TERM) to find the right entry. This is usually done through libraries like curses and then sends the matching escape sequences. To note, the database belongs to the operating system, not WezTerm, but it does determine the information WezTerm receives from programs [4].

Messaging

Pseudo terminal is used as the biggest communication. This is a channel between WezTerm and the actual program it runs, which is typically a shell. Only on Windows, the program goes through the ConPTY feature which works as a kind of extra terminal emulator sitting between WezTerm and the program [4]. The program sends a single stream of UTF-8 text that is mixed with escape sequences that control the position of the cursor, colours and the placements of many objects like the window tiles [6]. Shells can also use escape sequences to send WezTerm data about themselves, which is known as shell integration. These are called the Operating System Command. OSC 7 is the current folder it is meant for new tabs to open in the same place and OSC 133 is where the prompts happen, like the input and output displays [7]. Finally, the wezterm cli command lets scripts on the outside to control a running WezTerm, that means the scripts can start programs, send text, and can get back a list of open split up tabs and the text shown in them [8].

Networks

For the networks, there are two keyways the WezTerm can connect to another machine. A Multiplex is where the WezTerm can connect to a copy of WezTerm running on another remote machine either over SSH or through an encrypted TLS connection [3]. SSH is another way to connect, and it uses the normal SSH settings and keys to log into the remote machine that means with each new tab it allows another channel in the same connection [9]. With TLS WezTerm uses SSH once to start the remote side and gets a security certification that then reconnects automatically if the link happens to be interrupted [3]. Both have a pro and con and that is the plain SSH works anywhere without extra setup, while multiplexing takes more work to set up but keeps sessions alive. With connecting and multiplexing, multiplexing does need WezTerm installed on the remote machine too. If the connection gets interrupted, a TLS connection can connect back and pick up the session right where the user left off [3].

4 Use Cases

4.1 Use Case 1: Connecting to a Remote Multiplexer

One of WezTerm's most distinctive features is that the multiplexer can run separately from the GUI. A user can connect to a multiplexer on another machine, close their window, and later reconnect

to the same sessions. To begin a session, a domain must be set up in the Lua configuration. A domain is a distinct set of windows and tabs that can be reached through a Unix socket, SSH, or TLS [3].

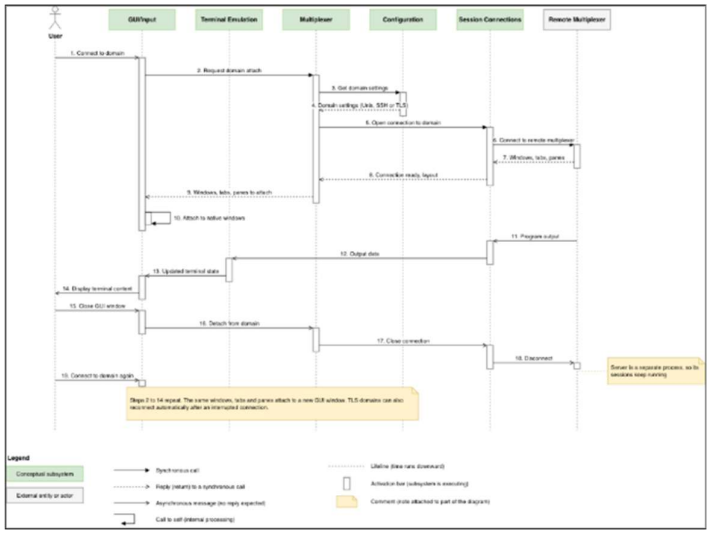


Figure 3: Sequence diagram for connecting to, disconnecting from, and reconnecting to a remote multiplexer

Figure 3 shows the flow. GUI/Input passes the user's connect request to the Multiplexer, which reads the domain's settings from Configuration and asks Session Connections to open the link. The remote server replies with the domain's windows, tabs, and panes, and the Multiplexer hands these to GUI/Input to attach to native windows [3]. From there, Terminal Emulation interprets program output, and GUI/Input draws it, just like a local session.

Closing the GUI ends the connection, but the sessions keep running because the multiplexer server is a separate process. When the user reconnects, the same steps repeat, and the sessions appear in a new GUI window. TLS domains can also reconnect on their own after an interruption [3].

4.2 Use Case 2: Editing the Configuration

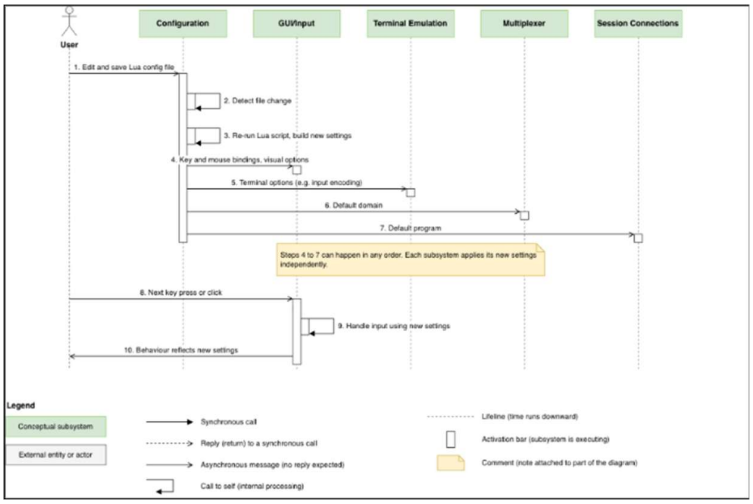


Figure 4: *Sequence diagram for editing the configuration and reloading it automatically*

This use case shows how the Configuration subsystem differs from the others. Rather than being a step in the path terminal data travels, it affects several subsystems at once. Here, a user with a running

WezTerm session changes a setting in the Lua configuration file, and the change applies without a restart.

In Figure 4 above, the user edits and saves the file, for example, to change a key binding or a visual option. WezTerm watches the config file it loaded, and when the file changes, it reloads the configuration automatically [1]. The file is a Lua script that returns the configuration to WezTerm [1]. Settings can also be passed on the command line [1].

Once the new configuration is loaded, the other subsystems use it. GUI/Input uses the updated key and mouse bindings and visual options. Terminal Emulation uses any updated terminal options, such as how keyboard input is encoded. The Multiplexer and Session Connections use the updated defaults, such as the default domain or program, when sessions are created. The next time the user types or clicks, GUI/Input handles it using the new settings.

5 Conclusion

Wezterm is a terminal emulator and multiplexer with many dependencies between its subsystems and external interfaces. GUI/Input relies on the multiplexer for session structure and on Terminal Emulation for interpreted terminal state. Multiplexer relies on terminal emulation and session connections to maintain pane state and create or join sessions, terminal emulation uses session connections to communicate with a pane's program, and configuration provides values to parts of the application. Separating responsibility organizes changes and can help with organizing change; however, dependencies between subsystems will allow changes or failures in one subsystem to impact the behavior of other parts of WezTerm.

In our research and analysis, we have shown that Wezterm's subsystems are closely connected to one another. These subsystems work together to accomplish incredible things like GUI/input which allows user input to cause operations in the terminal environment, Multiplexer which manages the organization of terminal sessions, Terminal emulation which interprets data, Session Connection which manages the connection between Wezterm and its environment, and Configuration which controls how things behave. Learning about these subsystems and their dependencies, and evolution showed us how adaptable Wezterm is. Contributors have added and will continue to add and update various features and fix problems that have been discovered as Wezterm continues to change and grow. Our next report will focus on the concrete architecture of Wezterm, where we will expand on the conceptual architecture presented and aim to expand upon the adaptability shown in this report.

6 Lesson Learned

Documentation that is oriented towards the end-user makes it difficult to recover an architecture; this is due to the fact that documentation describes features in a more direct manner than describing the relationships between those features. For future work, we will collect data for each subsystem and its dependencies while the model is being created. We will establish terminology at an earlier point and verify representative use-cases prior to finalization of our diagrams. By doing so, we will identify conflicting assumptions much sooner and therefore minimize revisions during integration.

Separating ordinary terminal input (separate from session management) allowed us to distinguish between data flow and control flow; this also prevented all interactions from being forced into the same subsystem sequence. We also see that WezTerm has multiplexing built into the terminal itself. This gives a richer, more integrated set of features for the user, but makes the system more complex and harder to maintain.

From a research standpoint, this project revealed how difficult it is to put together an architecture when the available documentation is user-oriented rather than architect-oriented. We had to navigate through the wezterm.org pages, the "what is a terminal" page, and the demo video to get a coherent

conceptual picture of the system. Because these sources describe the software from different angles, we had to cross-check them to make sure the details agree, to ensure our report was consistent. This experience showed how much the quality of documentation affects the ability of third parties to understand the system.

7 Data Dictionary

- **GUI/Input [Conceptual subsystem]:** Handles user input and displays terminal windows, tabs, panes, and their content.
- **Multiplexer [Conceptual subsystem]:** Organizes terminal sessions into windows, tabs, panes, domains, and workspaces.
- **Terminal Emulation [Conceptual subsystem]:** Interprets program output, maintains terminal state, and encodes input for running programs.
- **Session Connections [Conceptual subsystem]:** Manages communication with local or remote terminal processes and multiplexer servers.
- **Configuration [Conceptual subsystem]:** Supplies settings governing input, presentation, terminal behaviour, domains, and program selection.
- **Multiplexing domain [Session grouping]:** Groups related terminal sessions within a local or remote environment.
- **Pane [Terminal abstraction]:** Represents a terminal session, including its connection, associated processes, screen state, and scrollback.
- **PTY - Pseudo-terminal [Operating system interface]:** Connects WezTerm to a local Unix program through a terminal interface.
- **Scrollback [Retained terminal state]:** Stores earlier output beyond the visible screen for scrolling or searching.
- **Terminal escape sequence [Terminal protocol data]:** Encodes instructions such as moving the cursor or changing text attributes.

8 Naming Conventions

- **Mux (Multiplexer):** Shorthand appearing in documented names such as MuxWindow, distinguishing a multiplexer-managed window from a native GUI window.
- **SSH (Secure Shell):** The protocol used for remote terminal connections and as a communication channel to a remote WezTerm multiplexer.
- **TLS (Transport Layer Security):** The protocol securing TCP connections between WezTerm multiplexer clients and servers.
- **TCP (Transmission Control Protocol):** The network transport used by TLS multiplexer connections.
- **IME (Input Method Editor):** An input mechanism used to compose text before completed input is delivered for terminal processing.
- **OSC (Operating System Command):** A category of terminal escape sequences. The report discusses OSC 7 for reporting the current directory and OSC 133 for identifying shell prompt and command boundaries.
- **CPU/GPU (Central Processing Unit/Graphics Processing Unit):** Processing units associated with Software rendering and GPU-accelerated OpenGL or WebGpu rendering, respectively.
- **POSIX (Portable Operating System Interface):** The operating system standards referenced when describing local default shell selection.

9 References

[1] W. Furlong, "Configuration," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/config/files.html
[2] W. Furlong, "Features," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/features.html
[3] W. Furlong, "Multiplexing," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/multiplexing.html
[4] W. Furlong, "What is a terminal?," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/what-is-a-terminal.html
[5] Wezterm, “Commits”, GitHub. [Online]. Available: https://github.com/wezterm/wezterm/commits/main/README.md?after=cab25161054c50fd6c705db4ceefef0f1e5a9575+69. [Accessed: October 2, 2026].
[6] W. Furlong, "Escape sequences," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/escape-sequences.html
[7] W. Furlong, "Shell integration," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/shell-integration.html
[8] W. Furlong, "wezterm cli," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/cli/cli/index.html
[9] W. Furlong, "SSH," Wez's Terminal Emulator. Accessed: Oct. 04, 2026. [Online]. Available: https://wezterm.org/ssh.html
[10] W. Furlong, “Object: Pane,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/config/lua/pane/
[11] W. Furlong, “Workspaces / Sessions,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/recipes/workspaces.html
[12] W. Furlong, “Keyboard Concepts,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/config/keyboard-concepts.html
[13] W. Furlong, “Keyboard Encoding,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/config/key-encoding.html
[14] W. Furlong, “SplitPane,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/config/lua/keyassignment/SplitPane.html
[15] W. Furlong, “Scrollback,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/scrollback.html
[16] W. Furlong, “front_end,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/config/lua/config/front_end.html
[17] W. Furlong, “Launching Programs,” Wez's Terminal Emulator. Accessed: Oct. 1, 2026. [Online]. Available: https://wezterm.org/config/launch.html
[18] Wezterm, “How can the community help support WezTerm's next steps?”, GitHub. [Online]. Available: https://github.com/wezterm/wezterm/discussions/7845. [Accessed: October 2, 2026].
[19] W. Furlong, “Contributing to wezterm”, GitHub. [Online]. Available: https://github.com/wezterm/wezterm/blob/main/CONTRIBUTING.md. [Accessed: October 2, 2026].
[20] Wezterm, GitHub. [Online]. Available: https://github.com/wezterm/wezterm. [Accessed: October 2, 2026].
[21] Wezterm, GitHub. [Online]. Available: https://github.com/wezterm/wezterm/discussions. [Accessed: October 2, 2026].
[22] W. Furlong, “Contributing”, Wez’s Terminal Emulator. [Online]. Available: https://wezterm.org/contributing.html. [Accessed: October 2, 2026].
[23] Wezterm, “Change Log”, Wez’s Terminal Emulator. [Online]. Available: https://wezterm.org/changelog.html. [Accessed: October 2, 2026

10 AI Collaboration Report

10.1 AI Member Profile and Selection Process

Our group used two AI teammates for this project:

- **ChatGPT-5.6 Luna (OpenAI)** was our research assistant. It can search the web and link to current sources. WezTerm is actively developed, so we needed up-to-date information rather than a model relying only on its training data, which could be outdated or wrong.
- **Claude Sonnet 5.5 (Anthropic)** was our tutor and reviewer. It explains concepts clearly in plain language and can read images and diagram files, which we needed for our sequence diagram sketches. We also wanted a model that would teach us new tools rather than hand us finished answers, since we had to fully understand and edit our own work.

10.2 Tasks Assigned to the AI Teammate

Task	AI Model	Reasoning
Finding initial sources	ChatGPT	WezTerm's documentation, GitHub, and discussions contain a lot of material. AI can scan it quickly and point us to the pages worth reading.
Brainstorming subsystem boundaries	ChatGPT	We were unsure how to split WezTerm's features into subsystems. Talking ideas through with AI helped us test our reasoning, while the final decisions stayed with the group.
Drafting sequence diagrams	Claude	diagrams.net was new to us. A tutorial and first drafts based on our paper sketches shortened the learning curve without taking the design work away from us.
Reviewing diagrams against the rubric	Claude	A second reviewer helps catch missing requirements, such as diagram legends.
Spelling and grammar check	ChatGPT	AI handles basic proofreading well and catches small mistakes that are easy to miss in our own writing.

10.3 Interaction Protocol and Prompting Strategy

Instead of naming one primary prompt engineer, each member worked with the AI directly on their own section. Everyone already had experience writing prompts, and the person writing a section understood its content best and could judge whether the output was useful. Relaying everyone's requests through one person would have caused delays and lost context. To keep our work consistent, we agreed on three shared rules. First, every prompt gave the AI context about the project, including that we were studying WezTerm's conceptual architecture and the names of our five subsystems. This kept the AI's terminology consistent with the rest of the report. Second, the AI was used to explain, organize, and check ideas, never as a source of facts. Third, when learning something new, we asked the AI to take on a professor persona so that it taught us instead of doing the work for us.

Our sequence diagram prompt went through the most iteration. Our first version only asked for a step-by-step diagrams.net tutorial, which gave us broad instructions that did not follow the UML style we needed. We then attached the UML lecture slides so the output would follow the course's notation, and finally added the professor persona, which made the tutorial more hands-on. The final prompt was: "Pretend you are a professor teaching a student the diagrams.net software for the first time. Give a step-by-step tutorial on how to use this software using the sequence diagram attached as

an example. Ensuring the style follows that of the UML diagrams format in the lecture slides also attached."

10.4 Validation and Quality Control Procedures

We never treated AI output as correct on its own. Each type of output had its own check.

Sources. AI tools can suggest sources that do not exist, are outdated, or do not support the claim they are linked to. We opened every suggested link, confirmed it came from a reliable origin such as WezTerm's official documentation or GitHub repository, and read the relevant part to confirm it contained the information we needed. No AI-suggested source was cited without passing this check.

Technical claims. The AI sometimes made confident claims about WezTerm that the documentation did not support. Every technical statement had to be confirmed in the official documentation before it went into the report, and anything we could not confirm was left out.

Diagrams. We checked each draft against the notation in Professor Adams' UML lecture slides and against our five subsystems. This caught real errors. For example, Claude's drafts sometimes used dashed arrows for forward calls, even though dashed arrows should only show replies. We corrected every draft by hand until it was clear and matched the course's UML style.

Writing. The author of each section reviewed every grammar or spelling change before accepting it, to make sure the AI had not changed the meaning.

We also watched for the AI forgetting instructions from earlier in a conversation, which sometimes led to incorrect answers. When this happened, we restated the context and asked again.

10.5 Quantitative Contribution to Final Deliverable

Contribution	AI share
Written report text	~1% (basic spell and grammar check)
Sequence diagrams	~3% (first drafts, then heavily reworked)
Source discovery	~3% (found starting sources, humans read and chose)
Review and editing	~2% (rubric gap checks)
Overall estimate	**~9%**

10.6 Reflection on Human-AI Team Dynamics

Overall, AI saved us time, but it also created new work. Starting from a focused list of sources was faster than reading through many unhelpful web pages, and the diagrams.net tutorial shortened the learning curve with new software. However, verifying every source, claim, and diagram took real effort, and iterating on each prompt to get a useful and relevant output took more time than expected.

AI also influenced how we made decisions. As a sounding board, it helped us reason through how to divide WezTerm into subsystems. As a reviewer, it caught small gaps, such as a missing legend for our dependency arrows. The final decisions always stayed with the group.

Our only disagreement was over which AI to use. Some members were more familiar with ChatGPT and others with Claude, and people had different opinions about which one was better for which tasks. Rather than forcing one choice, we used both and trusted each member to pick the right tool for their own section. This is also part of why we ended up with two AI teammates instead of one.

The biggest lesson we learned is that every AI output must be checked, since the AI's confidence did not match its accuracy. We also found that specific prompts work much better than vague ones. Including our subsystem names and the section we were writing gave far more useful results. In future projects, we plan to give clear instructions from the start, verify everything the AI produces, and use AI to support our thinking rather than replace it